

Go Symbolization Insights

gopclntab · DWARF · Tooling

7 July 2026

Agenda

- One Binary, Two Tables - gopclntab vs. DWARF
- gopclntab Internals - format and PC to symbol resolution
- Stripping Go Binaries - what survives, what you lose
- Demos

One Binary, Two Tables

.debug_info / DWARF - the standardised debugging format

- Stripped by go build flag `-ldflags="-w"`
- Used by: Delve, GNU addr2line, perf
- Contains: types, variables, inlined call trees, precise source locations
- Standardised and any external tool can parse it reliably
- Sections: `.debug_info`, `.debug_line`, `.debug_frame`, `.debug_loclists`, ...

One Binary, Two Tables

.gopclntab — the Go Program Counter → Line Table

- Present in all Go binaries, including fully stripped ones
- Required by the runtime: panic traces, reflect, pprof unwinding
- Contains: function names, file/line tables, SP-delta for stack unwinding, inline trees
- Format is Go-specific and changed in Go 1.2, 1.16, 1.18, 1.20

gopclntab Internals

The (simplified) table is a flat binary blob in the ELF section `.gopclntab`.

```
-----  
Header          magic, ptrSize, quantum, nfunc, nfiles and  
                one offset field per table below  
-----  
funcnametab     null-terminated names  
-----  
cutab           per-CU index -> filetab  
-----  
filetab         source file name strings  
-----  
pctab           pcvalue byte sequences  
                (sp-delta, file, line, ...)  
-----
```

Defined as `pcHeader` in `src/runtime/symtab.go`.

gopclntab Internals

How external tools, like GNU addr2line and most eBPF/userspace profilers, do PC → Symbol lookups

```
PC (uintptr)
-> sort.Search(functab) 0(log n): entry with entryPC <= PC
-> _func{ nameOff, pcfileOff, pclnOff, ... }

nameOff    -> funcnametab[nameOff] -> "pkg.Func"
pcfileOff  -> pcvalue walk (deltas) -> filetab[idx] -> "file.go"
pclnOff    -> pcvalue walk (deltas) -> line number

-> Symbol{ Func, File, Line }
```

gopclntab Internals

runtime.FuncForPC and how the runtime does PC → Symbol lookups

```
PC (uintptr)
-> bucket := (PC - minpc) / 4096      direct index, 0(1)
-> subbkt := (PC - minpc) % 4096 / 256  direct index, 0(1)
-> idx     := runtime.findfuncbucket[bucket] + subbucket offset
-> linear scan forward in functab from idx until entry covers PC
-> _func{ nameOff, pcfileOff, pclnOff, ... }

-> Symbol{ Func, File, Line }
```

Stripping Go Binaries

What each flag actually removes:

```
# DWARF off, symbol table stays
go build -ldflags="-w" -o myapp ./...

# Symbol table off, DWARF stays
go build -ldflags="-s" -o myapp ./...

# Both off
go build -ldflags="-s -w" -o myapp ./...
```

What survives after each option:

flag	gopclntab?	DWARF?	.symtab?
-w	✓ stays	✗ gone	✓ stays
-s	✓ stays	✓ stays	✗ gone
-s -w	✓ stays	✗ gone	✗ gone

One more thing: -trimpath

-trimpath affects what gopclntab/DWARF stores for file paths

```
# Absolute source paths embedded in gopclntab:  
go build -o demo .  
  
# Panic output:  
main.handlePanic()  
    /home/alice/src/gosym-demo/main.go:128  
  
# Module-relative paths in gopclntab:  
go build -trimpath -o demo .  
  
# Panic output:  
main.handlePanic()  
    github.com/example/gosym-demo/main.go:128  
  
# Combined: smallest binary, clean paths, still full panic traces:  
go build -ldflags="-s -w" -trimpath -o demo .
```

-trimpath removes build-machine paths from gopclntab and DWARF when present.

Demo: Comparing Section Sizes

Build a minimal program, compare with and without debug info:

```
cat > /tmp/demo.go << 'EOF'
package main
import "fmt"
func add(a, b int) int { return a + b }
func main()           { fmt.Println(add(40, 2)) }
EOF

go build -o /tmp/demo /tmp/demo.go # full debug
go build -ldflags="-w" -o /tmp/demo_nodwarf /tmp/demo.go
go build -ldflags="-s -w" -o /tmp/demo_stripped /tmp/demo.go

# Section sizes with readelf
readelf -S /tmp/demo* | grep -E 'File:|\.gopclntab|\.debug_info|\.symtab'

# Binary sizes
ls -lh /tmp/demo*
```

Demo: go tool addr2line

go tool addr2line resolves a program counter to file and line number.
Despite the name, it reads gopclntab (via debug/gosym) — **not** DWARF.

```
# The GNU addr2line is a *different* tool -- it
# needs DWARF, and gives up once DWARF is stripped:
addr2line -f -C -e ./myapp_stripped 0x004a1b3c
??
??:0

# Interactive: type hex addresses, get file:line back
go tool addr2line ./myapp
0x004a1b3c
main.processRequest
/src/myapp/server.go:42

# Pipe a list of addresses (e.g. from a crash log)
printf '0x004a1b3c\n0x004a1c88\n' | go tool addr2line ./myapp

# Still resolves function + file:line on the fully stripped binary –
# no DWARF, no .symtab, just gopclntab
printf '0x004a1b3c\n' | go tool addr2line ./myapp_stripped
```

Key Takeaways

gopclntab is immortal

- Survives all stripping. The Go runtime requires it. Go binaries are always symbolizable.

DWARF is optional but powerful

- Strip with `-w` to save binary size. You lose: Delve, perf/DWARF unwinding
- Keep DWARF in debug builds; strip only for production

Recommendation for production

```
go build -trimpath -o myapp .                # full DWARF, clean paths
objcopy --only-keep-debug myapp myapp.debug    # save DWARF (+ symtab)
objcopy --strip-debug --add-gnu-debuglink=myapp.debug myapp  # ship myapp
```

Ship `myapp` and archive `myapp.debug` alongside the release for later full-DWARF debugging¹²

Thank you